

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

Train images shape: (60000, 28, 28)

Train labels shape : (60000,)

Test images shape : (10000, 28, 28)

Test labels shape : (10000,)

4. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Preprocessing performed: loading Fashion-MNIST, displaying sample images, flattening images to a 784-length vector, normalizing pixel values to [0, 1], and converting labels into one-hot vectors.

Before preprocessing:

x_train: (60000, 28, 28) dtype: uint8

After preprocessing:

x_train_flat: (60000, 784) dtype: float32

y_train_oh : (60000, 10)

5. Model Construction

The baseline MLP was constructed with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Model Summary

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650
=====		
Total params: 109,386 (427.29 KB)		
Trainable params: 109,386 (427.29 KB)		
Non-trainable params: 0 (0.00 B)		

The model was compiled with the Adam optimizer, Categorical Cross Entropy loss, and Accuracy as the evaluation metric, and trained for 20 epochs with a batch size of 32.

6. Source Code

The complete source code for this experiment is available at:

https://github.com/tonypanda7/DeepLearning_Lab

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, automated hyperparameter optimization was performed using **RandomizedSearchCV** together with the SciKeras wrapper.

Search space explored:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

First, a baseline model was trained using a basic MLP configuration. Next, we defined the hyperparameter search space, based on which RandomizedSearchCV was used to find the optimal set of hyperparameters. The search consisted of 5-fold cross-validation (with 20 candidate configurations and 100 total fits) and was employed to determine the optimal set of hyperparameters, after which the model was retrained and evaluated on the test set, comparing its performance to the baseline model.

8. Perceptron Experiments

The plots depict the decision boundary line, which slowly rotates and shifts based on each misclassified sample as the algorithm updates the weights. As such, the last plot shows the line managed to separate the only positive point (1, 1) from the rest of the negative points, proving that the algorithm was able to converge on a linearly separable problem..

8.1 AND, OR, and NOT Gates

A perceptron was trained on the AND, OR, and NOT gates using the perceptron learning rule with $lr = 1$. All three gates are linearly separable, and the perceptron converged successfully for each.

--- AND Gate ---

```
Update 1: w=[0. 0.], b=-1
Update 2: w=[1. 1.], b=0
Update 3: w=[1. 1.], b=-1
Update 4: w=[1. 0.], b=-2
Update 5: w=[2. 1.], b=-1
Update 6: w=[2. 0.], b=-2
Update 7: w=[1. 0.], b=-3
Update 8: w=[2. 1.], b=-2
Update 9: w=[1. 1.], b=-3
Update 10: w=[2. 2.], b=-2
Update 11: w=[2. 1.], b=-3
Converged weights: w=[2. 1.], b=-3
```

--- OR Gate ---

```
Update 1: w=[0. 0.], b=-1
Update 2: w=[0. 1.], b=0
Update 3: w=[0. 1.], b=-1
Update 4: w=[1. 1.], b=0
Update 5: w=[1. 1.], b=-1
```

Converged weights: $w=[1. \ 1.]$, $b=-1$

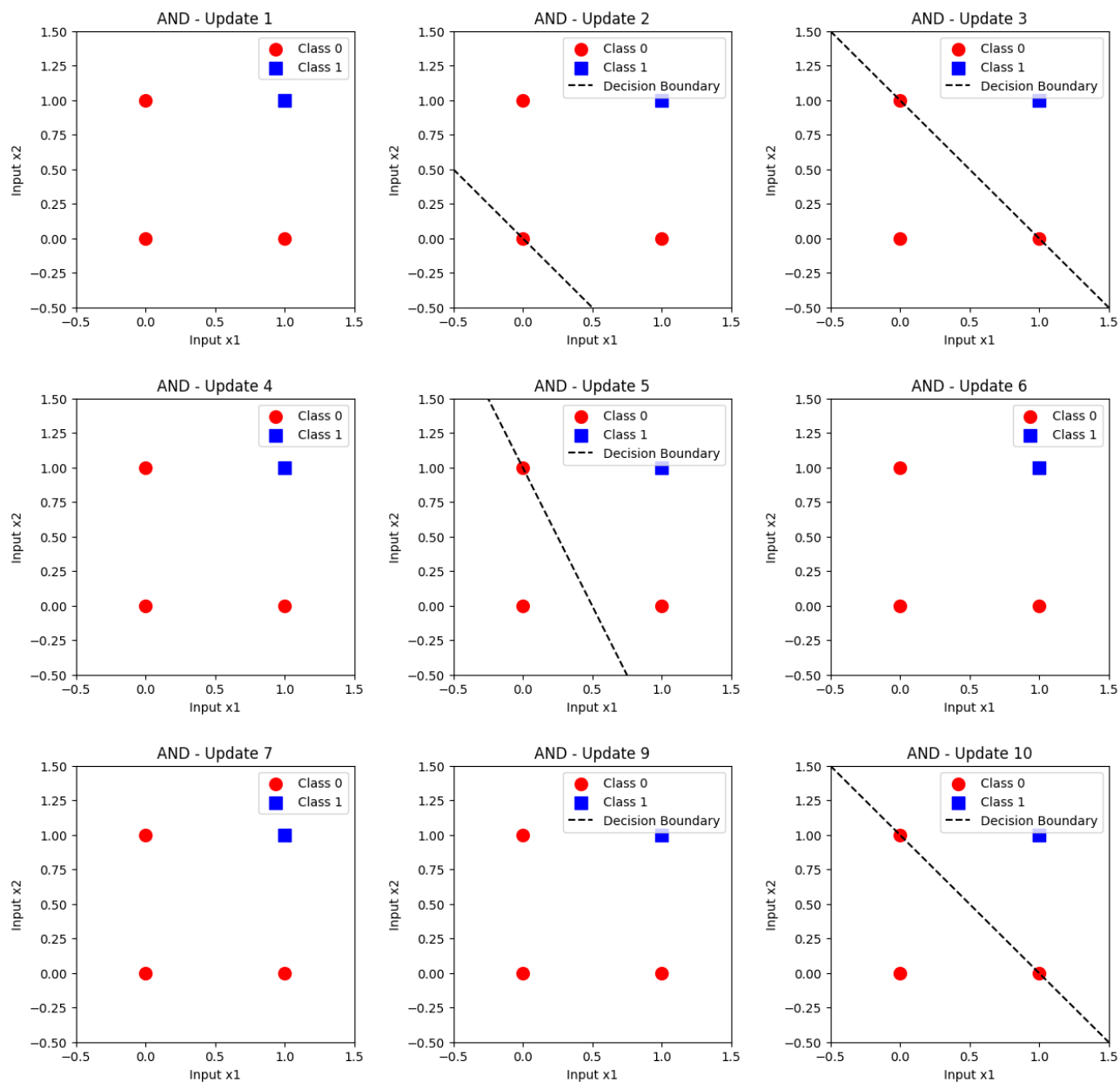
--- NOT Gate ---

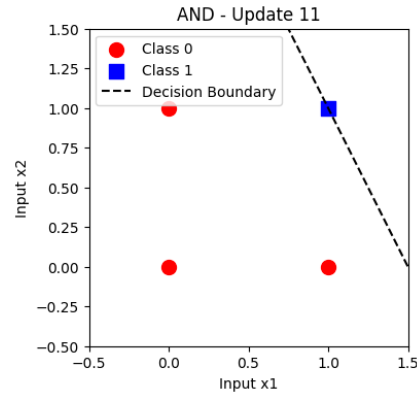
Update 1: $w=[-1.]$, $b=-1$

Update 2: $w=[-1.]$, $b=0$

Converged weights: $w=[-1.]$, $b=0$

The decision boundary of the AND gate perceptron evolves across successive weight updates as shown below.

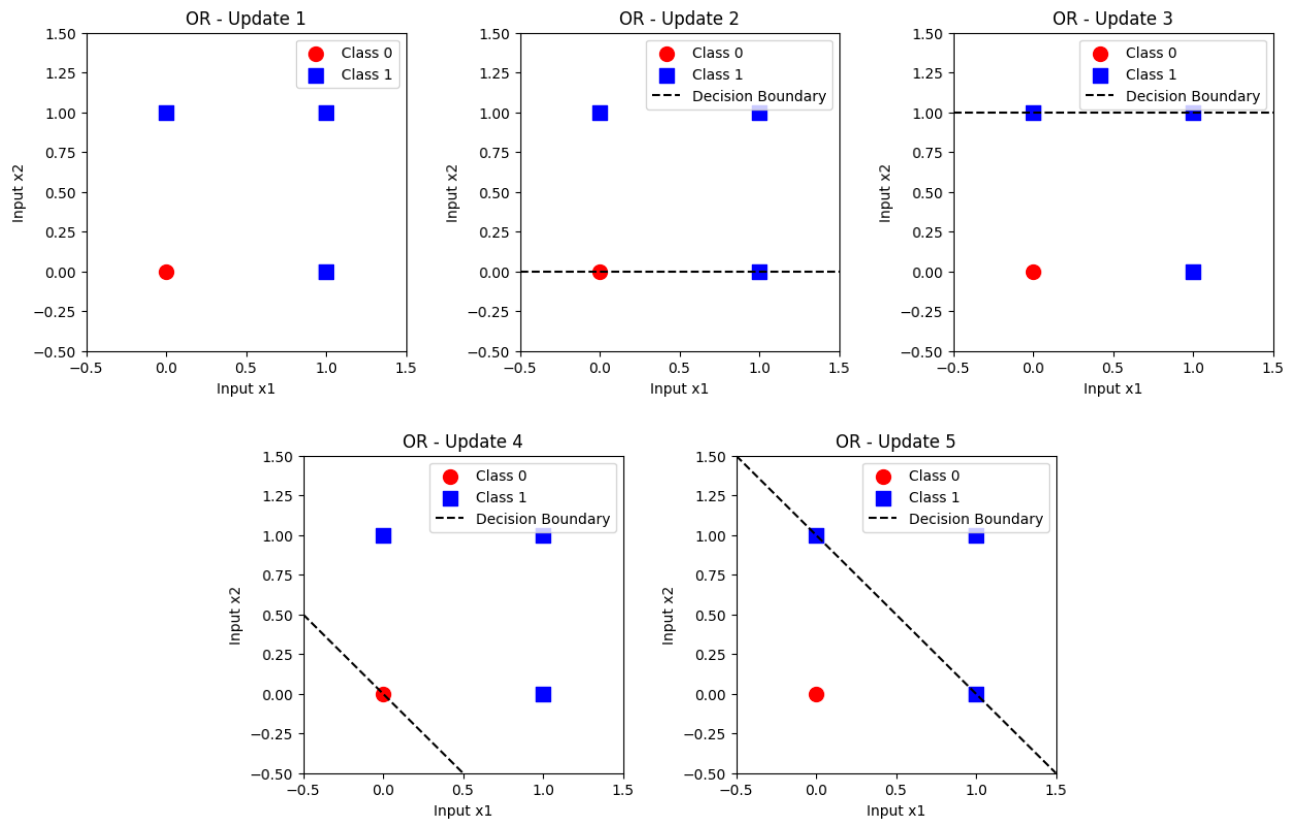




Interpretation: The plots depict the decision boundary line, which slowly rotates and shifts based on each misclassified sample as the algorithm updates the weights. As such, the last plot shows the line managed to separate the only positive point (1, 1) from the rest of the negative points, proving that the algorithm was able to converge on a linearly separable problem.

8.1b OR Gate

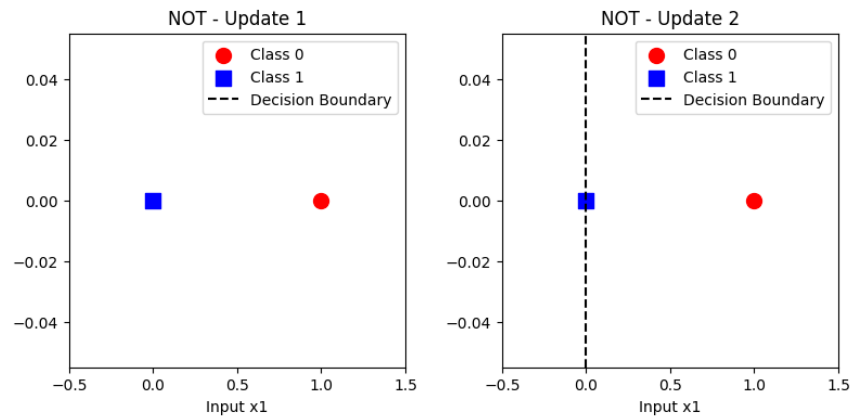
The OR gate decision boundary similarly evolves across weight updates until all points are correctly classified.



Interpretation: The OR gate boundary required fewer updates than the AND since 3 out of 4 input points were in Class 1. This means that the updates were needed for only 1 point which was the origin (0,0). The last boundary correctly classified the point at origin (0,0) as Class 0 and all the other three points as Class 1.

8.1c NOT Gate

For the single-input NOT gate, the decision boundary is a vertical line separating the two possible inputs, 0 and 1.



Interpretation: After the first update, the boundary line was not yet present since only the bias was adjusted; by the second update the boundary settled at $x_1 = 0$, having correctly placed input 0 in Class 1 and input 1 in Class 0.

8.2 XOR Gate – Non-Convergence

The same perceptron learning rule was applied to the XOR gate, which is not linearly separable. The perceptron was run for 50 epochs (198 weight updates) and failed to converge. It oscillillated between a few weight vectors without ever converging to a set of weights that would give zero classification error.

Ran 198 updates without hitting zero errors | perceptron never converges.

Update 1: $w=[0. 0.]$, $b=-1$

Update 2: $w=[0. 1.]$, $b=0$

Update 3: $w=[-1. 0.]$, $b=-1$

Update 4: $w=[-1. 1.]$, $b=0$

Update 5: $w=[0. 1.]$, $b=1$

Update 6: $w=[-1. 0.]$, $b=0$

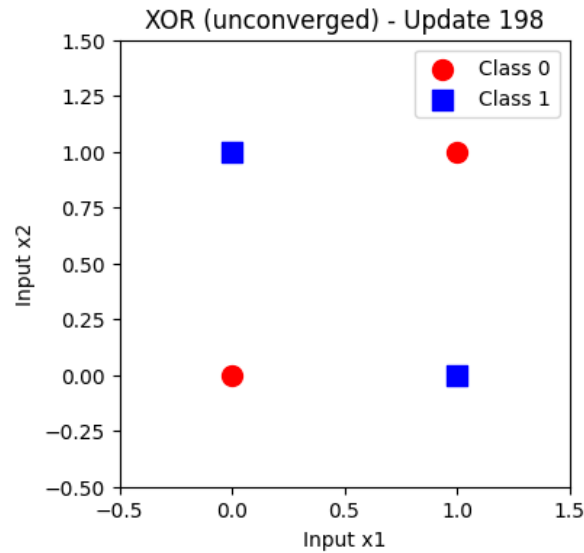
...

(pattern of weights repeats cyclically through

$w=[-1. 0.]$, $w=[-1. 1.]$, $w=[0. 1.]$, $w=[0. 0.]$ with no epoch achieving zero errors)

...

Update 198: $w=[-1. 0.]$, $b=0$



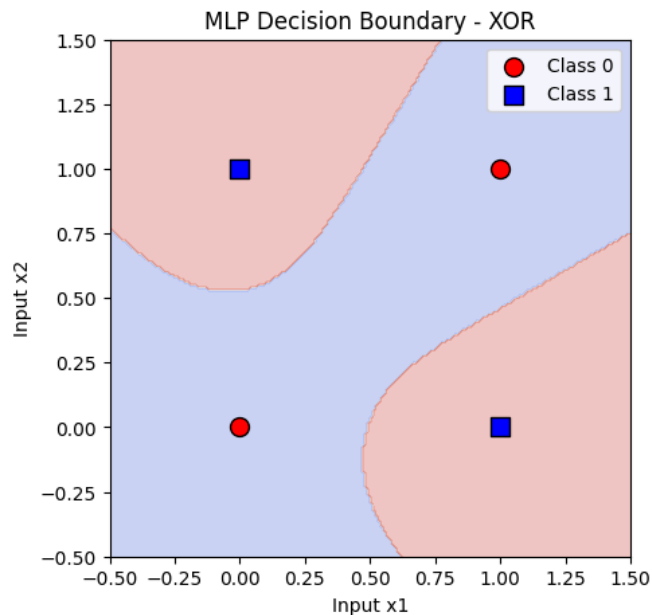
Interpretation: Because there is no straight line that can separate the XOR classes, the perceptron simply loops over several possible weight vectors, each of which incorrectly classifies at least one data point. The final result shown in Update 198 does not even have a decision boundary line (a.k.a. classification line), since the weight vector at that update step was $w = [-1, 0]$, which cannot form a line with any slope. This results in both Class 1 points lying on opposite sides of any classification line, a manifestation of the linear separability problem of a single-layer perceptron.

8.3 XOR Gate – Solved with a Multi-Layer Perceptron

To overcome the linear-separability limitation, a two-layer MLP (`MLPClassifier`, one hidden layer of 4 tanh units) was trained on the same XOR data.

Predictions: [0 1 1 0]

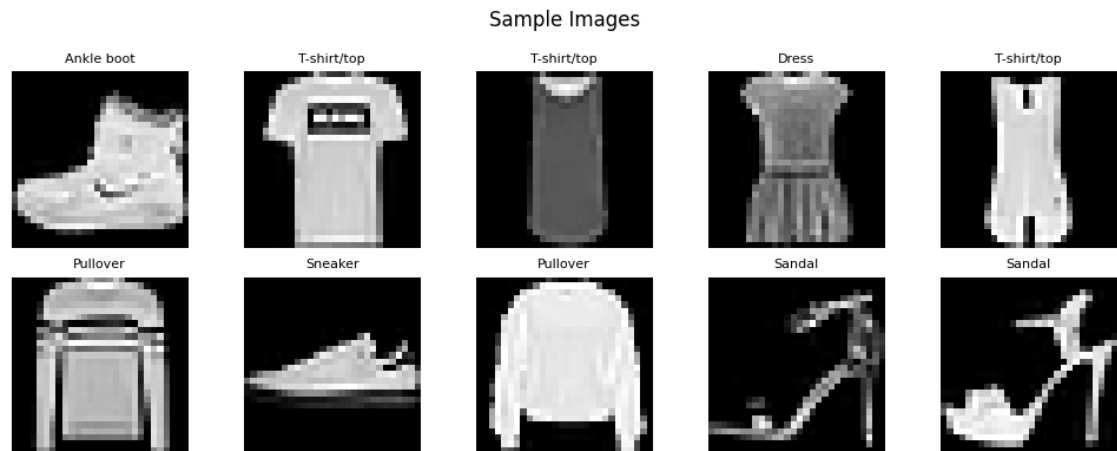
Accuracy: 1.0



Interpretation: By introducing a single hidden layer with non-linear activation functions, the MLP was able to form a curved, non-linear boundary that perfectly (100

9. Mandatory Plots

9.1 Sample Images



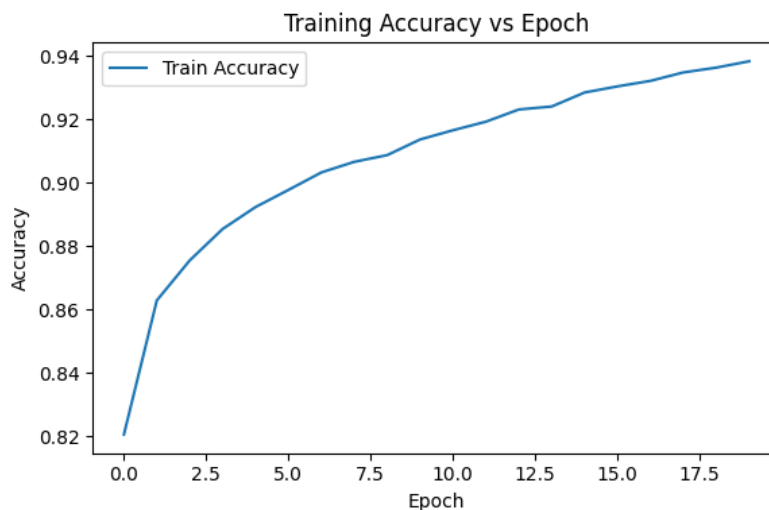
Interpretation: The sample grid displays one representative image per randomly drawn training example, covering categories such as T-shirts, dresses, pullovers, sneakers, and sandals. The images confirm that the dataset consists of low-resolution (28×28) grayscale garment photographs suitable for flattening into 784-dimensional feature vectors.

9.2 Class Distribution



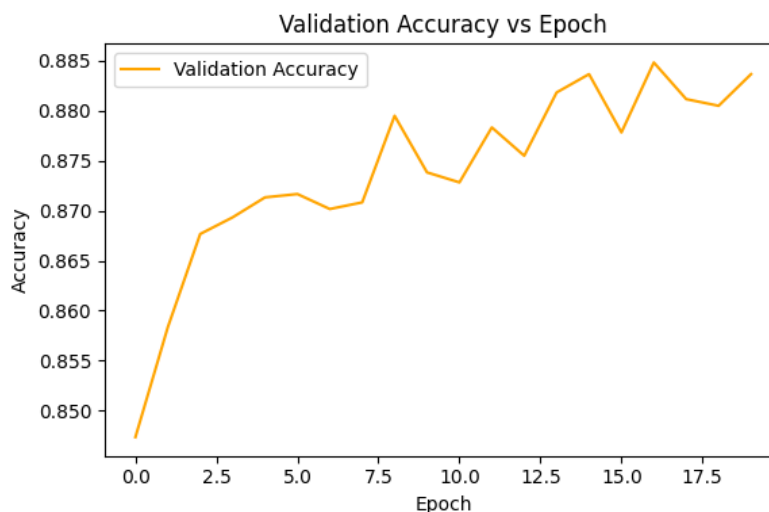
Interpretation: All ten classes contain exactly 6,000 training images each, so the training set is perfectly balanced. This means no class-weighting or resampling is required, and any accuracy differences observed later stem from the intrinsic visual difficulty of a class rather than data imbalance.

9.3 Training Accuracy vs Epoch



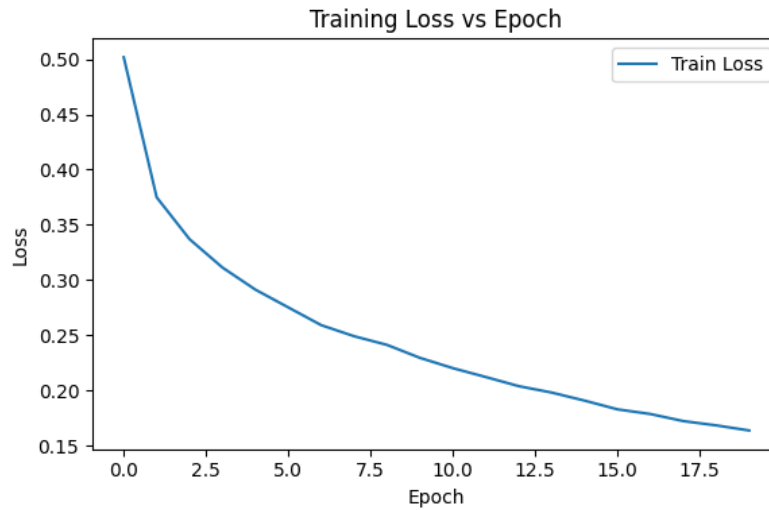
Interpretation: Training accuracy rises sharply during the first few epochs and then increases steadily, reaching about 93.8% by epoch 20. The smooth, monotonically increasing curve indicates that the model is consistently fitting the training data better with each epoch.

9.4 Validation Accuracy vs Epoch



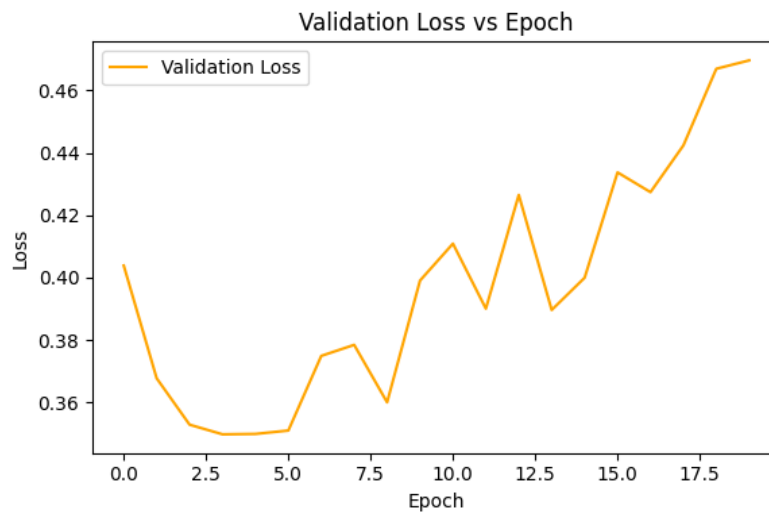
Interpretation: Validation accuracy improves quickly in the first few epochs and then plateaus around 87–88%, with visible fluctuation after epoch 8. The gap that opens up between training and validation accuracy in later epochs is an early indicator of overfitting.

9.5 Training Loss vs Epoch



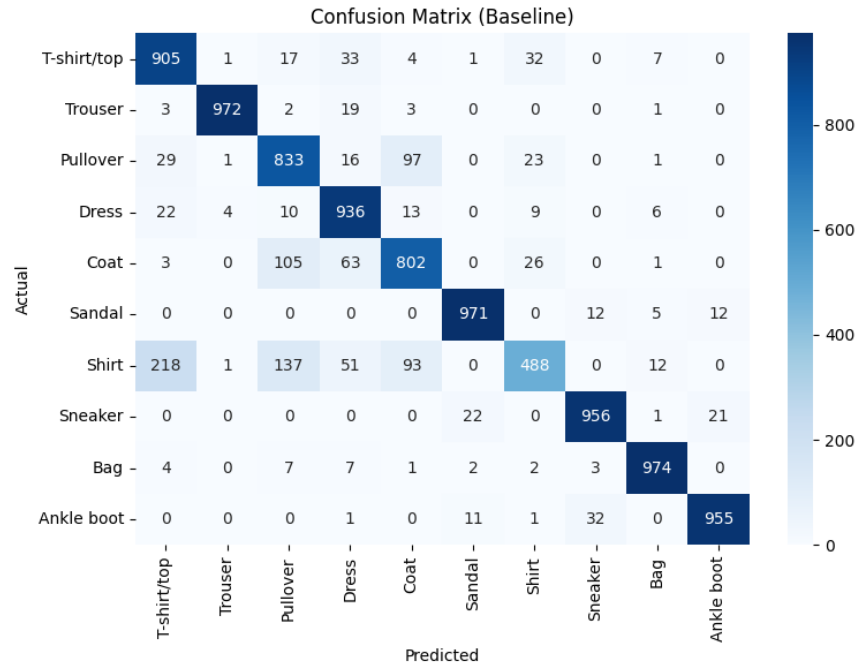
Interpretation: The loss decreases consistently from 0.50 to 0.16 during the training phase for 20 epochs. It appears that there is no overfitting or oscillation, which implies that the Adam optimizer converges correctly on the training set.

9.6 Validation Loss vs Epoch



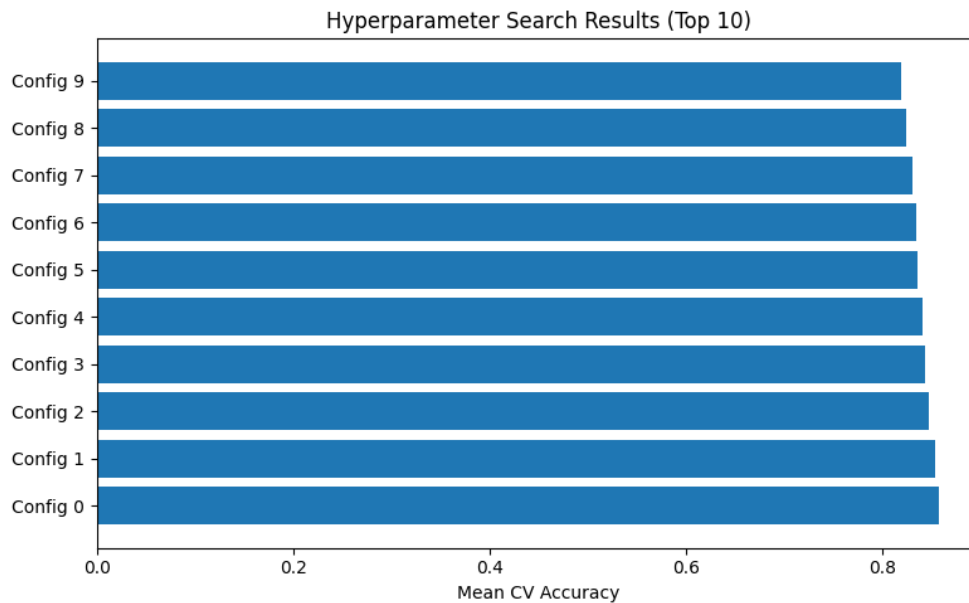
Interpretation: The loss on the validation set decreases until about epoch 4-5 (thus reaching approx. 0.35) and then starts increasing until almost 0.47 in epoch 20, while the training loss continues its decreasing trend. This means that after several epochs of training, our model has started to overfit the training set.

9.7 Confusion Matrix



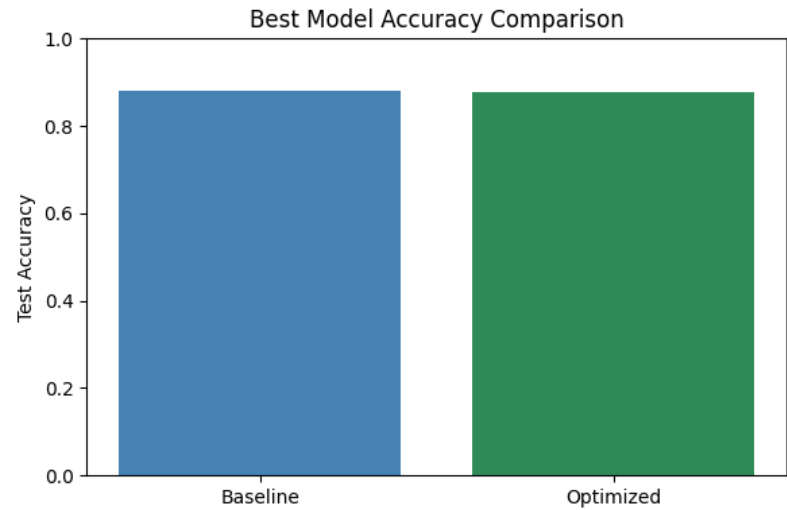
Interpretation: Most classes like Trouser, Sandal, Bag, and Ankle boot have high precision, reaching 900+ correctly identified words per 1000 words. The biggest challenges to identification are the classes of upper clothing: Shirt and T-shirt/top, Pullover, and Coat, which are often confused with each other.

9.8 Hyperparameter Search Results



Interpretation: The top 10 configurations from the randomized search all have mean cross-validation accuracies of approximately 0.82 to 0.86, with Config 0 (the winning configuration) only slightly edging out the others. In other words, the search procedure didn't find any significantly better configurations than the one that ended up winning the competition. This is probably because once you've got a good enough architecture and optimizer, small variations in the hyperparameters don't matter as much.

9.9 Best Model Accuracy Comparison



Interpretation: The bar chart comparing the baseline and optimized model’s test set accuracies is nearly impossible to read since the bars are practically on top of each other. This suggests that the baseline hyperparameters were already fairly close to the optimal values since the two models’ accuracies were nearly identical (around 0.88).

10. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSProp
Activation Function	Tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8576
Testing Accuracy	0.8787

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8792	0.8787
Precision	0.8820	0.8792
Recall	0.8792	0.8787
F1-score	0.8748	0.8773
Training Time (s)	114.95	148.46

11. Discussion and Conclusion

Discussion. The hyperparameters were optimized using the RandomizedSearchCV with 5-Fold Cross Validation using 20 sampled candidates (with 100 total fits) from the 8-Dimensional space. The optimal configuration found was a 3-hidden layer network with Tanh activation, with 128 Neurons per hidden layer. The optimizer was RMSProp with the learning rate of 0.001, the Batch size of 32, dropout of 0.2, and the training epochs of 30. It achieved mean cross-validation accuracy of 0.8576.

However, on the test set, the optimized model had accuracy of 0.8787, compared to the baseline of 0.8792, and F1-score of 0.8773, which was only slightly better than the baseline F1-score of 0.8748. Therefore, it could be stated that the optimization did not lead to significant performance gains on the test set, although the optimization procedure took significantly longer than the baseline one. Specifically, 3-layer network with 128 Neurons each takes roughly 148 seconds to train, compared to roughly 115 seconds of the baseline. Moreover, in terms of importance, the optimizer and the learning rate seem to affect the training process much stronger than the number of layers and the number of Neurons per layer. This can be seen from the fact that training processes with learning rates of 0.1 or optimizer SGD took much longer on cross-validation than RMSProp with the learning rate of 0.001. On the other hand, the number of layers and Neurons per layer did not have nearly as big effect, once the optimizer and learning rate were fixed.

The RandomizedSearchCV performs much better than the exhaustive GridSearchCV in this case, since it samples the given search space and selects the best model among the ones sampled. Given the fact that GridSearchCV would require thousands of models to be trained for an 8-dimensional search space compared to only 20 candidates in RandomizedSearchCV, it becomes obvious why the former would not be used in this case. Although the exhaustive search would not leave any uncertainty about the presence of a local optimum, it would require way too much computational power for such a high-dimensional search space, while the Randomized Search has enough flexibility to skip some of the irrelevant parameters and save computational time and effort.

Conclusion. A baseline MLP (784-128-64-10) trained for 20 epochs attained 87.9

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.